民泊ブログ自動化パイプライン
手順書

運用マニュアル｜担当者向け

プロジェクト: content-automation-pipeline / minpaku-blog

発行: 2026年5月

リポジトリ: github.com/hiro-57/content-automation-pipeline

民泊ブログ自動化パイプライン 手順書

> このドキュメントは民泊ブログの記事生成・公開を半自動化するシステムの**運用マニュアル**です。
> 担当者がこれ1つを読めば、日常運用・トラブル対応・改善まで一通りできるように書かれています。

📑 目次

- 0. はじめに
- 1. 全体アーキテクチャ
- 2. 日常運用ガイド（最重要）
- 3. 知識ベース運用
- 4. プロンプト改善ガイド
- 5. パイプライン詳細
- 6. 初期セットアップ（新環境構築時）
- 7. コスト・運用費
- 8. 設計判断の記録
- 9. 今後の検討事項
- 10. トラブルシューティング
- 11. 用語集
- 12. 付録

---

0. はじめに

このシステムは何か（一言で）

> **キーワードを与えると、人間以上の品質を狙ったブログ記事を半自動で生成し、WordPress に下書き投稿するシステム**

「半自動」のポイント:
- AI が**ドラフトと評価レポートを生成**する
- 人間が**実体験を加筆して最終公開**する
- AI 単体では出せない「**実体験 × 網羅性 × 品質**」の組合せで人間ライターを上回ることを狙う

誰が・何を運用するか

役割	主な作業	頻度
担当者（あなた）	キーワード補充、記事レビュー・実体験追記、WP公開、知識ベース更新	毎日〜週1
AI（Claude API）	記事生成、競合分析、評価レポート生成	担当者の指示で起動
WordPress	公開・SNS拡散（Jetpack 経由）	自動

全体の流れ（鳥瞰図）

```
[担当者]
   │ Ahrefs でキーワード調査 → CSV エクスポート
   ▼
inbox/ に CSV 配置
   │ python replenish_keywords.py
   ▼
Google スプレッドシート（キーワード一覧）
   │ python main.py を実行（毎日）
   ▼
┌──────────────────────────────────────┐
│  ① Claude が記事生成（〜1分）              │
│  ② バックアップを outputs/ に保存          │
│  ③ Claude が web_search で評価            │
│  ④ 評価レポートを outputs/ に保存          │
│  ⑤ Claude が記事に「加筆マーカー」を埋込    │
│  ⑥ Flux でサムネ背景生成 + HTML文字入れ     │
│  ⑦ WP メディアにサムネアップロード          │
│  ⑧ WordPress に下書き投稿（アイキャッチ付き）│
│  ⑨ スプレッドシート「処理済」更新           │
└──────────────────────────────────────┘
   │
   ▼
[担当者] WP 編集画面で:
   - アイキャッチ画像が設定されているのを確認
   - 黄色いマーカーボックスを順次対応:
     ・ 実体験エピソードを記入
     ・ そのマーカーブロックを削除
   - 数字・固有名詞のファクトチェック
   ▼
公開（WordPress）
   ▼
[Jetpack プラグイン] が SNS 自動拡散
   - Facebook / LinkedIn（無料）
   - X（X Premium 加入時のみ）
   - note・はてな は手動コピペ
```

重要な前提

- ⚠️ **AI は実体験を捏造しないように設計済み**。記事は「業界の専門家視点」で客観的に書かれる
- ⚠️ **公開前に必ず人間が編集する**。AI ドラフトの直接公開は品質と E-E-A-T（後述）の観点で禁止
- ⚠️ **数字・法律・固有名詞は人間が必ず検証する**。AI は時々ハルシネーション（事実の捏造）を起こす
- 💡 **すべての操作は Git で履歴管理**されている。問題があれば過去のバージョンに戻せる

---

1. 全体アーキテクチャ

ファイル / フォルダ早見表

```
projects/minpaku-blog/
│
├── 手順書.md                        ← ★ いまこのファイル
│
├── main.py                          ← メインの実行スクリプト（毎日これを動かす）
├── replenish_keywords.py            ← Ahrefs CSV → スプレッドシート転記
├── extract_kb.py                    ← 過去記事から voice_guide / industry_facts を自動抽出
├── make_pdf.py                      ← 手順書.md → 手順書.pdf 変換ツール
│
├── .env                             ← API キー類（⚠️ Git管理外・絶対に他人に見せない）
├── .env.example                     ← .env の雛形（Git管理対象）
│
├── google-credentials.json          ← Google サービスアカウント鍵（⚠️ Git管理外）
├── requirements.txt                 ← Python パッケージ一覧
├── README.md                        ← プロジェクトの簡単な説明
│
├── steps/                           ← パイプラインの各ステップ
│   ├── claude.py                    ← Claude API ラッパー（記事生成）
│   ├── evaluate.py                  ← 評価ロジック（web_search で競合比較）
│   ├── annotate.py                  ← 加筆マーカー埋め込み（黄色ボックス挿入）
│   ├── thumbnail.py                 ← サムネ生成（Flux + HTML文字入れ）
│   ├── sheets.py                    ← Google Sheets 読み書き
│   ├── wordpress.py                 ← WordPress 投稿（メディアアップロード含む）
│   └── dify.py                      ← 旧版（参考用に残置・実は使ってない）
│
├── prompts/                         ← AI のプロンプト（Git管理）
│   ├── generate_article.md          ← 記事生成プロンプト
│   ├── evaluate_article.md          ← 評価プロンプト
│   ├── annotate_article.md          ← 加筆マーカー埋め込みプロンプト
│   ├── generate_thumbnail_prompt.md ← サムネ画像プロンプト生成
│   ├── extract_voice_guide.md       ← voice_guide 抽出プロンプト
│   └── extract_industry_facts.md    ← industry_facts 抽出プロンプト
│
├── templates/                       ← HTMLテンプレート
│   └── thumbnail.html.tpl           ← サムネ文字入れ HTML
│
├── knowledge/                       ← 知識ベース（RAG）
│   ├── README.md                    ← 知識ベースの説明
│   ├── voice_guide.md               ← ブランドボイス（K3a）
│   ├── target_audience.md           ← ターゲット読者像
│   ├── industry_facts.md            ← 業界の事実集（K3b）
│   ├── personal_experiences.md      ← 運営者の実体験（K1・最重要）
│   ├── unique_perspectives.md       ← 独自視点・差別化の切り口
│   └── past_articles/               ← 過去のベスト記事（K2）
│       └── README.md
│
```

```
├── inbox/                          ← Ahrefs CSV 投入フォルダ
│   ├── README.md                   ← 使い方
│   └── processed/                  ← 取り込み済みCSVの自動退避先
│
├── outputs/                        ← AI 生成物の自動保存先（⚠️ Git管理外）
│   ├── YYYY-MM-DD_HHMMSS_キーワード.md             ← 生成記事
│   ├── YYYY-MM-DD_HHMMSS_キーワード.annotated.md   ← 加筆マーカー入り
│   ├── YYYY-MM-DD_HHMMSS_キーワード.evaluation.md  ← 評価レポート
│   ├── YYYY-MM-DD_HHMMSS_キーワード.thumbnail-bg.jpg ← Flux 生成の背景
│   └── YYYY-MM-DD_HHMMSS_キーワード.thumbnail.png  ← 文字入れ後のサムネ
│
└── .venv/                          ← Python 仮想環境（⚠️ Git管理外）
```

データの流れ

```
[1] キーワード補充（週1）
    Ahrefs UI → CSV → inbox/ → replenish_keywords.py → Google Sheets

[2] 記事生成（毎日）
    Google Sheets → main.py
                    ├→ Claude API → outputs/*.md
                    ├→ Claude API + web_search → outputs/*.evaluation.md
                    └→ WordPress REST API → 下書き投稿
                    └→ Google Sheets を「処理済」に更新

[3] 人間編集（毎日）
    outputs/*.evaluation.md（参照）+ WP 編集画面（編集）→ 公開

[4] 拡散（自動）
    WP公開 → Jetpack → Facebook / LinkedIn / X
    note・はてな は担当者が手動コピペ
```

関与する外部サービス一覧

サービス	用途	月額目安	担当者がやること
Anthropic Claude API	記事生成・評価・加筆マーカー・サムネプロンプト	$40〜80（記事100本想定）	API キー管理
Replicate（Flux 1.1 Pro）	サムネ背景画像生成	$4〜5（記事100本想定）	API トークン管理・課金監視
Google Cloud（Sheets API）	キーワード管理	無料枠内	Service Account 管理
Google スプレッドシート	キーワード一覧	無料	列の整合性維持
Ahrefs（Lite プラン）	キーワード調査	$129/月（既存）	UI でエクスポート
WordPress	記事公開・メディア管理	サーバー代	編集・公開
WordPress App Password	API 認証	無料	漏えい時に再生成
Jetpack（WPプラグイン）	SNS自動拡散	無料（X除く）	初期設定のみ

2. 日常運用ガイド（最重要）

担当者が実際にやることまとめ

毎日やること

時間	作業	コマンド／場所
朝	パイプライン実行（記事生成→評価→WP下書き作成）	`python main.py`
昼〜夕	WP の下書きを開いて編集（評価レポート見ながら実体験追加）	WordPress 管理画面
編集後	WP で公開ボタンを押す	WordPress 管理画面
公開後	note・はてな に手動コピペ（任意）	各サイト

毎週やること

作業	頻度	コマンド／場所
Ahrefs でキーワード調査 → CSV エクスポート → inbox/ に配置	週1	Ahrefs UI + ファイル操作
キーワードをスプレッドシートに転記	週1	`python replenish_keywords.py`
評価で気づいた改善点を `personal_experiences.md` に追記	週1	テキストエディタ

毎月やること

作業	頻度	コマンド／場所
`industry_facts.md` を最新情報で更新	月1	テキストエディタ
Anthropic API の利用額確認	月1	https://console.anthropic.com/
過去記事のうちベスト1〜2本を `knowledge/past_articles/` に追加	月1	ファイルコピー
知識ベース更新後 → `python extract_kb.py` で再抽出を提案	月1	コマンド

コマンド早見表

> 全コマンドは `projects/minpaku-blog/` フォルダで実行する前提です。
> ターミナル（PowerShell or Git Bash）でこのフォルダに `cd` してから実行してください。

```
# パイプライン実行（記事1本生成）
.venv/Scripts/python.exe main.py

# Ahrefs CSV を取り込み
.venv/Scripts/python.exe replenish_keywords.py

# 過去記事から voice_guide / industry_facts を抽出（K2 充実後に実行）
.venv/Scripts/python.exe extract_kb.py

# Claude 単体テスト（main.py の生成部分だけ動かす）
.venv/Scripts/python.exe steps/claude.py "テストキーワード"

# 評価単体テスト（既存記事を評価する）
.venv/Scripts/python.exe steps/evaluate.py "キーワード" "outputs/記事ファイル.md"
```

1記事を公開するまでの完全フロー（チェックリスト）

```
□ ターミナルで projects/minpaku-blog/ に移動
□ .venv/Scripts/python.exe main.py を実行
□ ターミナル出力で以下を確認:
  □ [1/6] 記事生成完了（文字数・トークン数）
  □ [2/6] 評価完了（総合スコア）
  □ [3/6] 加筆マーカー埋め込み完了（マーカー数）
  □ [4/6] サムネイル生成完了（PNG / WPメディアID）
  □ [5/6] WP 下書き ID と編集 URL
  □ [6/6] スプレッドシート更新
□ WP 編集 URL をブラウザで開く
□ 編集画面で以下を順番に対応:
  □ 黄色いマーカーブロック（💡加筆ヒント）を上から順に対応:
    □ ヒントの内容を読む（実体験を加筆する場所と内容の指示）
    □ そこに自分の実体験・固有名詞・数字を書く
    □ ⚠️ 加筆が終わったら **黄色ブロック自体を削除**
  □ 数字・固有名詞・URL を全部チェック（ファクトチェック）
  □ タイトル・パーマリンクを最終調整
  □ アイキャッチ画像が自動設定されているか確認（必要なら差し替え）
  □ カテゴリー・タグを設定
□ プレビューで全体を確認:
  □ ⚠️ 黄色いマーカーが残っていないか必ず確認（公開後に出ないように）
  □ Ctrl+F で「加筆ヒント」を検索 → ヒット 0 件であること
□ 公開ボタンを押す
□ Google スプレッドシートを開いて article_url が記録されているか確認
□ outputs/ の .md / .annotated.md / .evaluation.md は手動で削除可（任意）
□ note・はてなブログに手動コピペ（任意）
```

⚠️ **黄色いマーカーを削除し忘れて公開しないこと**。これが最大の運用リスクです。プレビュー画面で必ず Ctrl+F「加筆ヒント」検索 → 0 件を確認してから公開してください。

---

3. 知識ベース運用

なぜ知識ベースが重要か

知識ベース（`knowledge/` フォルダ）は **Claude が記事を書く時に参考にする「あなたの会社の脳みそ」**です。ここを充実させるほど、生成記事の質と独自性が上がります。

```
知識ベースが空っぽ → AI 一般論的な記事
知識ベースが充実   → あなたの会社にしか書けない記事
```

ファイル構成と役割

ファイル	役割	充実度の目安	担当者の関与
`voice_guide.md`	ブランドボイス・トーン	必須	編集判断
`target_audience.md`	ターゲット読者像	必須	マーケ判断
`industry_facts.md`	業界の事実集（法律・データ）	推奨	リサーチ
`personal_experiences.md`	運営者の実体験・失敗談	**最重要**	運営者本人
`unique_perspectives.md`	独自視点・差別化の切り口	推奨	編集判断
`past_articles/*.md`	過去のベスト記事	あると劇的に質UP	編集判断

K1: personal_experiences.md（最重要）

人間超えの記事を作る最大の源泉です。

何を書く？

- 自分が運営して経験したトラブル・解決策
- ゲストとの実際のやり取り（具体的に）
- 収益・コストの実数
- 業者選びの基準
- 失敗から学んだ教訓

書き方のコツ

> ✅ **完璧な文章でなくてOK**。箇条書きの断片メモが何十個も並んでいる方が、Claude が記事化するときに使い

> やすいです。

```
### 開業時に起きたトラブル
- 保健所の届出で書類不備で2回突き返された。原因は消防設備の証明書の様式違い
- 内見後に「思ったより狭い」とキャンセルされた。Airbnb の写真撮影で気をつけるべきは広角レンズ過多
```

更新タイミング

- 新しいトラブルや学びがあった時に都度追記
- 月1回まとめて整理
- 記事を書きながら「この経験は使えそう」と気づいたら即追記

K2: past_articles/

過去のベスト記事を Claude が「書き方の見本」にするためのフォルダ。

入れるべきもの

- ✅ 自分が書いて手応えのあった記事
- ✅ アクセスが伸びた記事
- ✅ ブランドボイスが綺麗に出ている記事

入れない方がいいもの

- ❌ 古くて法律情報が現状と違う記事（混乱の元）
- ❌ AI 生成された記事（ループになって品質劣化）

ファイル名の付け方

```
2024-12_minpaku-osaka-guide.md
2025-03_minpaku-tax-handling.md
```

`YYYY-MM_テーマ.md` で並び順がわかりやすくなります。

推奨数

3〜10本。多すぎるとプロンプトが冗長化してコストが増えます。

K3a: voice_guide.md（ブランドボイス）

ブログ全体のトーンを定義する。担当者が編集判断で書きます。

定義する項目

- **基本トーン**: 敬体（です・ます調）/ 常体（だ・である調）
- **一人称**: 「私」「私たち」「当社」など
- **望ましい表現**: 数字を出す、体験談を入れる、等
- **避けたい表現**: AI clichés の禁止
- **用語ルール**: 「ホスト」or「運営者」、「ゲスト」or「宿泊客」など

サンプル

`knowledge/voice_guide.md` を参照（既にテンプレが入っています）。

K3b: industry_facts.md（業界の事実集）

ハルシネーション防止のため、検証済みの事実だけを集める。

必ず書くべきこと

- 法律名・施行日（住宅宿泊事業法 = 2018年6月施行 など）
- 営業日数の上限（180日以内）
- 主要プラットフォーム名と特徴
- 自治体の特殊条例
- 用語の正確な定義

更新タイミング

- 法律改正のニュースを見たら即更新
- 月1で観光庁の最新統計を確認
- Claude が古い情報で記事を書いたと気づいたら、その情報源を追加

extract_kb.py の使い方

K2（過去記事）から K3（voice_guide / industry_facts）を**自動で下書き生成**するツール。

使うタイミング

```
1. knowledge/past_articles/ にベスト記事が3〜5本以上揃った時
2. 過去記事を新しく追加した時（既存の K3 と統合する目的）
```

実行手順

```
# 1. 過去記事を knowledge/past_articles/ に配置
# 2. 抽出ツールを実行
.venv/Scripts/python.exe extract_kb.py
```

出力ファイル

- `knowledge/_proposed_voice_guide.md`
- `knowledge/_proposed_industry_facts.md`

両方ファイル名の頭に `_proposed_` がついています（提案の意味）。

反映の手順

```
1. _proposed_voice_guide.md を開いてレビュー
2. 違和感のある記述を修正
3. 既存の voice_guide.md と統合（or 上書き）
4. _proposed_voice_guide.md を削除
5. industry_facts.md も同様
6. Git に diff として記録（git add → commit）
```

---

4. プロンプト改善ガイド

なぜプロンプトを編集する？

Claude の出力品質はプロンプト次第。**評価レポートで「いつも同じ弱点」が指摘されたら、プロンプトを改善するチャンス**です。

ファイル一覧

```
prompts/
├── generate_article.md          ← 記事生成プロンプト（最重要）
├── evaluate_article.md          ← 評価プロンプト
├── extract_voice_guide.md       ← voice_guide 抽出プロンプト
└── extract_industry_facts.md    ← industry_facts 抽出プロンプト
```

generate_article.md の編集ガイド

構造

```
## SYSTEM PROMPT
[Claude の役割・絶対ルール・出力形式]

## USER PROMPT
[毎回変わる部分: キーワード + 知識ベース埋め込み]
```

編集ルール

- 1回の変更は**1つだけ**にする（A/B テスト原理）
- 変更前に必ずコミット（git で diff を残す）
- 変更後、必ずテスト実行して品質確認
- 悪化したらすぐ git で戻す

よくある改善パターン

評価で出る指摘	プロンプトでの対処
同じ表現を繰り返す	「同じフレーズを連続使用しない」を追加
表が少なくて読みにくい	「3項目以上の比較は必ず表で」を追加
結論が薄い	「最後に『次に取るべき行動』を必ず明記」を追加
検索意図と合わない	「読者は ○○ を求めて検索しているので △△ を中心に」と具体化

evaluate_article.md の編集ガイド

評価軸を変えたい時はここを編集。

軸の変更例

現在は6軸（網羅性・深さ・独自性・実用性・読みやすさ・検索意図適合）。

追加したい軸の例:
- `accuracy` （事実正確性） — 数字や法律の正確さを評価
- `actionable_steps` （具体的手順） — 「次にこうしよう」が明確か
- `originality_evidence` （独自性の根拠） — 単なる目新しさでなく根拠の強さ

注意

評価軸を変えると過去スコアと**比較不可能**になります。長期トレンドを追いたいなら軸はめったに変えない方が良いです。

改善ループの回し方

```
1. main.py を5〜10回実行（複数記事の評価結果を見る）
   ↓
2. 評価レポートに共通する弱点を特定
   例: 「3記事連続で『独自性スコアが低い』と指摘されている」
   ↓
3. generate_article.md に改善指示を追加
   例: 「他社が触れていない切り口を必ず1つ含める」
   ↓
4. 1記事生成して評価を見る
   ↓
5. 改善されていれば commit、悪化したら revert
```

---

5. パイプライン詳細

各ステップが具体的に何をしているか・どこで詰まるか。

5.1 キーワード調達（半自動）

全体の流れ

```
Ahrefs UI でキーワード調査
    ↓
フィルタ（KD・volume）で絞り込み
    ↓
Export → CSV (UTF-8) でダウンロード
    ↓
inbox/ にコピー
    ↓
python replenish_keywords.py
    ↓
Google スプレッドシートに転記（重複は自動スキップ）
    ↓
inbox/processed/ に CSV が自動移動
```

Ahrefs UI 操作の手順

1. **Ahrefs にログイン** → Keywords Explorer を開く
2. シードキーワード（例: `民泊` ）を入力 → 検索
3. 左メニュー 「**Matching terms**」 をクリック
4. 上部のフィルタ:
   - Volume: `100〜10000` など
   - KD: `30以下` など
5. **Export ボタン**:
   - Group by terms: ☐ チェックしない
   - Include top 10 positions from SERP for each keyword: ☐ チェックしない
   - Number of keywords: 全件（All）
   - Format: **CSV (UTF-8)** ← これ一択
6. ダウンロードした CSV を `inbox/` にコピー

replenish_keywords.py の動作

- `inbox/*.csv` を全部読み込む
- `Keyword` カラムだけ抽出
- スプレッドシートに既にあるキーワードはスキップ
- 新しいものだけ追加（status は空欄）

- CSV を `inbox/processed/` に移動

5.2 記事生成（Claude）

流れ

```
スプレッドシートから未処理キーワードを1件取得
    ↓
prompts/generate_article.md を読み込み
    ↓
knowledge/*.md を全部読み込み（プロンプトに埋め込む）
    ↓
Claude API 呼び出し（モデル: claude-sonnet-4-5）
    ↓
記事 Markdown を返す
    ↓
outputs/YYYY-MM-DD_HHMMSS_キーワード.md に保存
```

プロンプトキャッシュの仕組み

```
1回目の呼び出し:
   - System prompt + 知識ベース（全部）= キャッシュ作成 ~4,000 tokens
   - キーワード = 変動部分 ~50 tokens

2回目以降（5分以内）:
   - キャッシュヒット → 同じ内容は90%割引
```

これで月100記事でも生成コストは ~$15 に収まります。

コスト目安

項目	1記事あたり
入力（変動分）	~$0.001
キャッシュ作成（初回のみ）	~$0.015
出力	~$0.10
合計	**~$0.10〜0.15**

5.3 評価（Claude + web_search）

流れ

```
記事 Markdown
    ↓
prompts/evaluate_article.md を読み込み
    ↓
Claude API 呼び出し（web_search ツール付き・最大5回）
    ↓
Claude が裏で:
    - キーワードでGoogle検索
    - 上位5記事を読み込み
    - 提供された記事と比較
    - 6軸でスコアリング
    - JSON で評価結果を返す
    ↓
JSON をパース → Markdown レポートに整形
    ↓
outputs/YYYY-MM-DD_HHMMSS_キーワード.evaluation.md に保存
```

評価軸

軸	内容
comprehensiveness（網羅性）	上位記事のトピックをカバーしているか
depth（深さ・具体性）	数字・固有名詞・事例の密度
originality（独自性）	上位記事にない切り口があるか
actionability（実用性）	読者の次の行動が明確か
readability（読みやすさ）	文章のテンポ・視覚的整理
search_intent_fit（検索意図適合）	検索者が求める情報と合致するか

⚠️ E-E-A-T は採点対象外。代わりに「人間が追記すべきヒント」として出力されます。

コスト目安

項目	1記事あたり
入力（記事+競合5記事の本文）	~$0.40
出力（評価レポート）	~$0.06
web_search（4〜5回）	~$0.04
合計	**~$0.50**

JSON パース失敗時の挙動

`json-repair` ライブラリで自動修復を試みます。それでもダメなら**評価をスキップして WP 投稿は続行**します（評価は補助情報・ブロッカーではない）。

5.4 加筆マーカー埋め込み（annotate）

流れ

```
評価結果から human_action_hints を取得
    ↓
prompts/annotate_article.md を読み込み
    ↓
Claude API 呼び出し（記事 + ヒント を渡す）
    ↓
Claude が記事を1文字も変えずに、適切な位置に黄色のマーカーブロック挿入
    ↓
outputs/YYYY-MM-DD_HHMMSS_キーワード.annotated.md に保存
```

マーカーの形式

```
<blockquote style="background:#fff8dc;border-left:6px solid #f0ad4e;padding:12px;margin:14px 0;">
💡 <strong>加筆ヒント</strong>: <em>{ヒント内容を要約}</em><br>
⚠️ <small>このブロックは公開前に必ず削除してください</small>
</blockquote>
```

WP 編集画面では**黄色いボックス**として表示され、担当者が見つけやすくなります。

コスト目安

項目	1記事あたり
入力（記事 + ヒント）	~$0.02
出力（マーカー入り記事）	~$0.10
合計	**~$0.12**

失敗時の挙動

annotate が失敗してもパイプラインは続行（マーカー無しのクリーン版で WP 投稿）。

なぜマーカーを記事内に直接埋めるか

評価レポートに「H2『〇〇』の後に追加」と書いてあっても、担当者は WP 編集画面で:
1. レポートと WP を行き来

2. 該当の見出しを探す
3. 「セクションの後」のどこに入れるか自分で判断

という手間が発生します。マーカーが本文中の**正確な位置**に黄色ボックスで挿入されていれば、こうした作業が不要になります。

5.5 サムネイル生成（thumbnail）

流れ

```
記事タイトル + 記事冒頭 800 文字
    ↓
prompts/generate_thumbnail_prompt.md → Claude が英語プロンプトを作成
    ↓
Replicate (Flux 1.1 Pro) で 1200x630 (16:9) JPG 背景画像を生成
    ↓
templates/thumbnail.html.tpl に画像 + タイトル + ブランド名を流し込み
    ↓
Edge headless で PNG スクリーンショット（1200x630）
    ↓
WordPress REST API で /wp-json/wp/v2/media に POST
    ↓
返ってきた media_id を post 投稿時の featured_media に指定
```

使用ツール

- **画像生成**: Flux 1.1 Pro（Replicate API 経由）
- **文字入れ**: HTML+CSS を Microsoft Edge headless でスクリーンショット
- **フォント**: Google Fonts の Noto Sans JP（CDN ロード）

コスト目安

項目	1記事あたり
Claude（プロンプト生成）	~$0.01
Flux 1.1 Pro（画像生成）	~$0.04
WP メディアアップロード	$0
合計	**~$0.05**

失敗時の挙動

サムネ生成・アップロードのどこかで失敗してもパイプライン続行（**アイキャッチなしで WP 投稿**）。担当者が手動でアイキャッチを設定すれば良い。

デザインのカスタマイズ

`templates/thumbnail.html.tpl` を編集:
- 色: `#d97706` を変更 → ブランドカラーに合わせる
- フォントサイズ: `.title` の `font-size` を調整
- ブランド名: `MINPAKU JOURNAL` を steps/thumbnail.py の DEFAULT_BRAND で変更

`prompts/generate_thumbnail_prompt.md` を編集:
- 画像のテイスト（写真風 / イラスト風 / 抽象画 など）
- 構図の好み

5.6 WordPress 下書き投稿

流れ

```
記事 Markdown
    ↓
H1（# タイトル）を抽出 → WordPress の post title に
    ↓
残りの本文を Markdown → HTML 変換
    ↓
WordPress REST API（POST /wp-json/wp/v2/posts）
    ↓
status: "draft"（⚠️ 必ず下書き・自動公開しない）
    ↓
記事ID と編集URL を返す
```

認証

WordPress Application Password を使います。通常のログインパスワードでは API 動作しません。

設定方法

```
1. WP管理画面にログイン
2. https://[サイト]/wp-admin/profile.php を開く
3. 下にスクロール → 「アプリケーションパスワード」セクション
4. 名前を入力（例: minpaku-blog-pipeline）→ 新しいパスワードを追加
5. 表示された 24 文字のパスワードをコピー（1回しか表示されない）
6. .env の WP_APP_PASSWORD に貼り付け
```

セキュリティ

- アプリケーションパスワードはいつでも WP 管理画面から**取り消せる**
- 漏えいの可能性があれば即座に取り消し → 新しいものを生成
- 通常のログインパスワードとは別物なので、ログインアカウントは安全

5.7 人間レビュー・公開

ここが**最も重要**な部分です。AI ドラフトを「人間超え」に押し上げます。

加筆マーカー（黄色ボックス）の活用

WP 下書きには、Claude が**黄色のマーカーブロック**を本文中の適切な位置に挿入しています:

```
💡 加筆ヒント: 実際に届出を行った時のエピソード（窓口対応・所要時間など、200字程度）
⚠️ このブロックは公開前に必ず削除してください
```

担当者の作業はシンプルです:

1. WP 編集画面を上から下に読む
2. **黄色ブロックを見つけたら**:
   - その指示に従って、自分の実体験をその場所に書く
   - 終わったら**黄色ブロック自体を削除**
3. すべての黄色ブロックが削除されるまで繰り返し

これで「どこに何を書くか」の迷いがなくなります。

編集チェックリスト

```
□ WP 編集画面を開く
□ 上から順に黄色マーカーを処理:
  □ マーカー1: 指示に従って加筆 → ブロック削除
  □ マーカー2: 指示に従って加筆 → ブロック削除
  □ ...（マーカーの数だけ繰り返し）
□ 加筆内容のチェック:
  □ 具体的な数字が入っているか（「〜万円」「〜%」「〜年」）
  □ 固有名詞が入っているか（業者名・物件名・地名）
  □ 自分の実体験として書かれているか（「私の場合」「実際〜」）
□ ファクトチェック（AI が書いた部分）:
  □ 法律名・施行日が正確か
  □ 引用した統計が最新か
  □ URL が正しく機能するか
  □ 数字に間違いがないか
□ タイトル・パーマリンクの最終調整
□ アイキャッチ画像を設定
□ カテゴリー・タグを設定
□ 内部リンク（過去記事へのリンク）を追加
□ プレビューで全体確認:
  □ ⚠️ Ctrl+F で「加筆ヒント」検索 → ヒット 0 件を確認
  □ 黄色いボックスが残っていないか目視確認
□ 公開
```

⚠️ **最重要: 公開前に黄色マーカーが全部消えているか必ず確認**してください。残ったまま公開すると、読者に編集メモが見えてしまいます。

公開判断のガイドライン

評価スコア	推奨アクション
8.0以上	軽微な実体験追加で公開
7.0〜7.9	不足トピック対応 + 実体験追加で公開
6.0〜6.9	大幅な加筆 or 再生成検討
6.0未満	再生成（原因分析後に）

⚠️ スコアはあくまで参考。**人間の判断が最優先**です。

5.8 バックアップ管理

保存場所

```
outputs/
├── YYYY-MM-DD_HHMMSS_キーワード.md           ← 生成記事
└── YYYY-MM-DD_HHMMSS_キーワード.evaluation.md ← 評価レポート
```

Git 管理外

`outputs/` は `.gitignore` で除外されています。**Git にはコミットされません**。

削除タイミング

- 公開済みの記事のバックアップは**手動で削除可**（容量節約）
- ただし「公開後のトラブルで原文に戻したい」ケースのため**1ヶ月程度は保管推奨**

自動削除（未実装）

将来的に「WP で公開されたら outputs/ から自動削除」を実装する案があります（9. 今後の検討事項 参照）。

---

6. 初期セットアップ（新環境構築時）

新しい PC で動かす場合や環境が壊れた時の再構築手順。

必要なもの

- Windows 10 / 11 PC
- インターネット接続
- 以下のアカウント:
- GitHub（リポジトリ取得）
- Anthropic（Claude API）
- Google（Sheets / Cloud）
- WordPress（投稿先サイト）
- Ahrefs（キーワード調査）

手順

Step 1: Python 3.12 をインストール

```
winget install --id Python.Python.3.12 --scope user --accept-source-agreements --accept-package-agreements --
```

確認:

```
python --version
# → Python 3.12.x が表示されればOK
```

Step 2: リポジトリをクローン

```
cd C:\Users\[ユーザー名]\OneDrive\ドキュメント
git clone https://github.com/hiro-57/content-automation-pipeline.git
cd content-automation-pipeline\projects\minpaku-blog
```

Step 3: Python 仮想環境を作成

```
python -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Step 4: API キーの取得

4-1. Anthropic API キー

1. https://console.anthropic.com/ にログイン
2. **API Keys** → **Create Key**
3. 表示されたキー（`sk-ant-api03-...`）をコピー

4-2. Google サービスアカウント

1. https://console.cloud.google.com/ にログイン
2. プロジェクト作成（既存ならスキップ）
3. **Sheets API を有効化**: https://console.cloud.google.com/apis/library/sheets.googleapis.com
4. **サービスアカウント作成**: https://console.cloud.google.com/iam-admin/serviceaccounts
5. JSON キーをダウンロード → ファイル名を `google-credentials.json` に変更
6. `projects/minpaku-blog/google-credentials.json` に配置
7. JSON 内の `client_email` を確認 → そのメールアドレスでスプレッドシートを共有（編集者権限）

4-3. WordPress アプリケーションパスワード

1. WP管理画面 → ユーザー → プロフィール
2. 下にスクロール → **アプリケーションパスワード** セクション
3. 名前を入力（例: `minpaku-blog-pipeline`）→ 追加
4. 表示された 24 文字のパスワードをコピー（1回しか出ない）

Step 5: .env ファイル作成

`projects/minpaku-blog/.env.example` をコピーして `.env` を作成し、自分の値を埋める:

```
# Anthropic Claude API
ANTHROPIC_API_KEY=sk-ant-api03-xxxxxxxxxxxxxxxxxxxx
CLAUDE_MODEL=claude-sonnet-4-5

# Google スプレッドシート
KEYWORDS_SHEET_ID=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

# WordPress
WP_SITE_URL=https://your-site.com/
WP_USERNAME=admin
WP_APP_PASSWORD=xxxx xxxx xxxx xxxx xxxx xxxx
```

Step 6: スプレッドシートの準備

新規スプレッドシート作成 → 1行目を以下のヘッダーに:

keyword	status	article_url	published_at

URL から ID を取得して `.env` の `KEYWORDS_SHEET_ID` に設定。

Step 7: 動作確認

```
# 軽い動作確認（API キーチェック）
.venv\Scripts\python.exe -c "import anthropic, gspread; print('OK')"

# 実際にパイプラインを動かす（テスト用キーワードを1件 sheet に追加してから）
.venv\Scripts\python.exe main.py
```

エラーが出なければセットアップ完了。

---

7. コスト・運用費

月額試算（記事数別）

記事数/月	生成	評価	検索	加筆ﾏｰｶｰ	サムネ	合計
10本	$1.5	$5	$0.5	$1.2	$0.5	**約 $9（≒¥1,300）**
50本	$7.5	$25	$2.5	$6	$2.5	**約 $44（≒¥6,500）**
100本	$15	$50	$5	$12	$5	**約 $87（≒¥13,000）**
200本	$30	$100	$10	$24	$10	**約 $174（≒¥26,000）**

⚠️ 上記はパイプラインの API 利用料のみ。**Ahrefs（$129/月）・WordPress サーバー代は別途**。

コスト構造の内訳（1記事 = 約¥120〜130）

```
記事生成              : ¥22  ← Claude API（出力 ~5,000 tokens）
評価・出力            : ¥10  ← Claude API（出力 ~4,000 tokens）
評価・入力            : ¥60  ← 競合記事5本の本文取込
web_search             : ¥6   ← 検索 4〜5回
加筆マーカー埋め込み : ¥18  ← Claude API（記事再出力 ~6,000 tokens）
サムネ画像生成        : ¥6   ← Flux 1.1 Pro（Replicate 経由・$0.04）
サムネプロンプト生成 : ¥1   ← Claude API（軽量呼び出し）
               合計 : 約¥123
```

評価ステップが約5割を占めます。コスト削減したい場合は評価・マーカー埋め込み・サムネをスキップする選択肢もあります。

コスト最適化のコツ

1. プロンプトキャッシュを最大限活用

5分以内の連続実行はキャッシュヒットして90%割引。

やってはいけない: 1日に1回だけ実行（毎回キャッシュ作成からやり直し）。
理想: バッチで複数記事を連続生成。

2. 評価を毎回しない選択肢

質に余裕があるキーワードは評価をスキップして生成のみにすればコスト1/4。

実装方法: スプレッドシートに `skip_eval` 列を追加（要追加実装）。

3. モデル選択

- **Sonnet 4.6**（現状）: 質と価格のバランス
- **Haiku 4.5**: コスト1/3 だが質は2段階下がる（推奨しない）
- **Opus 4.7**: 質は最高、コストは3倍（重要記事のみ向け）

`.env` の `CLAUDE_MODEL` で切替可。

課金アップグレードの判断基準

「現在の半自動を完全自動化するか？」の判断ポイント。

Ahrefs API への移行

```
IF 月の記事生成数 < 50本
    → 半自動継続（現状維持）

ELIF 月の記事生成数 50〜200本
    → 月の手動作業時間を計測
    → 1.5時間/月超えたら Standard プランへの upgrade を検討
        ・Lite ($129/月) → Standard ($249/月) で +$120/月
    → それ未満なら半自動継続

ELIF 月の記事生成数 ≥ 200本
    → Standard or Advanced + API 自動化を強く推奨
        ・記事数増えるとスケール時の人為ミス・運用負荷が無視できない

ELIF 「最新キーワードを毎日反映したい」要件あり
    → 記事数に関わらず API 検討
```

⚠️ Ahrefs プラン料金は変動するので**契約時の最新価格を確認**してください。

X（Twitter）自動投稿

X API は無料プランでは投稿不可。

```
IF X 自動投稿が必要
    → X Premium ($8/月〜) 加入
    → WP の Jetpack プラグインに X アカウント連携
    → 記事公開時に自動投稿される
```

画像生成 AI

実装済み: Flux 1.1 Pro（Replicate 経由）。$0.04/枚で写真品質のサムネイル背景を生成。
詳細は 5.5 サムネイル生成 参照。

---

8. 設計判断の記録

「なぜこの仕組みになったか」を辿れる記録。**今後の改修・引継ぎ時の参考**になります。

採用した主要ツール・選択

採用1: Python + 直接 Claude API

採用理由

- ✅ プロンプトを Git で履歴管理可能
- ✅ コードレビュー・A/B テストが容易
- ✅ Claude の最新機能（プロンプトキャッシュ・web_search）を直接使える
- ✅ コスト透明（Anthropic Console で全部見える）

メリット

- 改善ループが回しやすい
- Claude の進化に直接乗れる
- 知識ベース・プロンプトを自由に管理

デメリット

- Python の知識が多少必要
- ビジュアルワークフロー UI がない
- リトライ・エラー通知などは自前実装

採用2: Google Sheets でキーワード管理

採用理由

- ✅ 既存のスプレッドシート資産を活用
- ✅ 担当者がブラウザで直接編集可能
- ✅ Google Sheets API は無料枠で十分
- ✅ サービスアカウントで安全に自動化

メリット

- 担当者の学習コスト 0
- 履歴・コラボ機能が標準で使える

デメリット

- API 呼び出し回数に上限あり（1日数百回まで・通常運用なら問題なし）
- 大量データには不向き（数千行で重くなる）

採用3: WordPress REST API + Application Password

採用理由

- ✅ WordPress 標準機能で追加プラグイン不要
- ✅ アプリケーションパスワードで安全
- ✅ 通常のログイン認証情報を漏らさない

メリット

- WP 管理画面から即座に rotate 可能
- パスワード漏えい時に即対応

デメリット

- WP 5.6 以降のみ対応
- HTTPS 必須

採用4: WordPress Jetpack で SNS 拡散

採用理由

- ✅ コードを書かずに WP 公開時の自動拡散
- ✅ Facebook / LinkedIn は無料
- ✅ note・はてな は手動コピペで運用回避

メリット

- メンテ不要
- 担当者がポチポチ設定するだけ

デメリット

- X は別途 Premium プラン必要
- note は API がないため対象外

却下した選択肢

却下1: n8n（ビジュアルワークフローツール）

検討理由

最初の依頼書（spec）が n8n のワークフロー JSON だった。

却下理由

- ❌ 月額費用が発生（self-host も保守必要）
- ❌ ワークフロー履歴が Git で管理できない
- ❌ コードレビューが事実上不可
- ❌ Python の方が柔軟・拡張可能

結論

「n8n でできることは Python でも全部できる、しかも安く長期運用できる」と判断。

却下2: Dify（AI ワークフロー SaaS）

検討理由

序盤は Dify free で記事生成していた。プロンプトを GUI で編集できる。

却下理由

- ❌ Dify free 枠のモデル制限で品質が頭打ち
- ❌ プロンプトを Git 管理できない（変更履歴・diff・blame・PR レビューが不可）
- ❌ コスト透明性が低い（Dify 課金 + 内部 LLM 課金の二重構造）
- ❌ 知識ベース（RAG）も Dify 内に閉じてしまい、コードと統合しにくい

結論

「同等品質で Claude API 直接の方が安くて柔軟」と判断 → 移行完了。
旧 `steps/dify.py` は参考用に残置中（次の整理タスクで削除予定）。

却下3: Ahrefs API（完全自動キーワード調達）

検討理由

「キーワード生成 → 絞り込み」全部を自動化したい。

却下理由

- ❌ 現在の Ahrefs Lite プランでは API アクセス不可
- ❌ Standard 以上にアップグレードすると月 +$120〜$320 のコスト増

- ❌ 月100記事程度なら半自動 CSV ワークフローで十分（手動 5〜10分/週）
- ❌ 投資回収しない

結論

「個人事業の規模ではコストが見合わない」と判断 → 半自動方式を採用。
ただし 月 200 記事以上に増えたら再検討（7. コスト・運用費 参照）。

却下4: note 自動投稿

検討理由

note は読者層が違うので拡散したい。

却下理由

- ❌ note 公式 API が存在しない
- ❌ Cookie ハック方式は note の利用規約違反の可能性
- ❌ アカウント凍結リスク
- ❌ 実装してもいつ壊れてもおかしくない

結論

「**note は手動コピペ**運用」に決定。1記事 2〜3分の作業で許容できる。

却下5: SNS 自動投稿コードを自前で書く

検討理由

X / Facebook / LinkedIn / はてな への自動投稿。

却下理由

- ❌ 各 SNS の API 仕様変更に追従する保守コスト
- ❌ X は無料プランで投稿不可（Premium 加入必要）
- ❌ WP の Jetpack プラグインで同じことができる

結論

WordPress Jetpack に任せる方針。コード保守ゼロで Facebook / LinkedIn は実現可能。
X は Jetpack + X Premium ($8/月) で実現。
はてなブログは API（AtomPub）が公式無料なので、将来的にコードで実装する余地あり（9. 今後の検討事項 参照）。

却下6: SEO 評価軸（旧版）→ search_intent_fit に改名

検討理由

「SEO 評価」を AI が見るべきか。

却下理由

- ❌ キーワード戦略・順位追跡は **Ahrefs の領域**
- ❌ AI が見るべき SEO は「検索意図適合」のみ
- ❌ 二重計測になり評価軸が混乱する

結論

「SEO」軸を `search_intent_fit` （検索意図適合）に改名して領域明確化。
- Ahrefs: キーワード選定・順位追跡（書く前後）
- AI 評価: 検索意図と中身の合致（書いた後）

却下7: E-E-A-T 評価軸（採点軸から外した）

検討理由

Google が SEO 評価で重視するから、点数化したい。

却下理由

- ❌ AI 生成段階では実体験ゼロのため**常に低スコア**
- ❌ 採点する意味が薄い（毎回 4〜5/10 で変動なし）
- ❌ 「人間が追記すべきヒント」の方が実用的

結論

E-E-A-T は採点軸から削除。代わりに `human_action_hints` で「人間が編集時に追記すべき箇所」を具体的に提示する形に変更。

採用5: Flux 1.1 Pro（Replicate）でサムネ画像生成 + HTML+CSS で文字入れ

採用理由（DALL-E 3 / Stability AI / Imagen 3 / Midjourney との比較結果）

- ✅ **写真品質**: 民泊・京町家など日本特有の風景を自然に描画
- ✅ **長文プロンプト理解**: Claude が記事から作る詳細プロンプトを正確に解釈
- ✅ **API シンプル**: Replicate のトークン1個追加だけ（Vertex AI のような追加プロジェクト設定不要）
- ✅ **コスト**: $0.04/枚 = 月100記事で $4

文字入れは Pillow ではなく HTML+CSS+Edge headless 採用

- Pillow 直書きだとデフォルトフォントで「ダサく」なりがち

- HTML/CSS なら Noto Sans JP・gradient・text-shadow など Web 品質の文字組が直接使える
- レンダリングは Microsoft Edge headless で（Windows 標準搭載・追加インストール不要）

却下した代替案

- **DALL-E 3**: 写真用途では絵画的になりがち、コスト同じなら Flux/Imagen 優位
- **Stability AI**: 同等品質に到達するためのプロンプト技術力依存が高い
- **Midjourney**: 公式 API 無し（Discord 連携のみ・自動化に不向き）
- **Imagen 3**: 日本文化解像度はやや上だが Vertex AI セットアップが面倒で実用差小

E-E-A-T を採点しない代わりの対策

E-E-A-T を採点しない代わりに、以下で品質を担保しています:

1. **人間編集が必須**: AI ドラフトを直接公開しない
2. `personal_experiences.md`: 実体験を Claude に学習させる
3. `human_action_hints`: 評価レポートで「実体験を加えるべき箇所」を具体提示
4. **ファクトチェックの運用**: 数字・法律は人間が必ず確認
5. **筆者情報の WP 設定**: WP のプロフィールに実名・経歴・資格を載せる（Google が読む）

---

9. 今後の検討事項

「やった方がいい」けどまだやってないこと。優先度と実装イメージ。

すぐやれる改善（半日〜1日）

1. 公開検知 → outputs/ 自動削除

何が変わる？

WP で記事を公開すると `outputs/` のバックアップが自動削除される。

メリット

- 容量節約
- バックアップが古くなる前に整理される

実装イメージ

```
# 新スクリプト: cleanup_published.py（毎日実行）
# 1. WP API で「最近公開された記事」一覧を取得
# 2. outputs/ に対応するバックアップがあれば削除
```

優先度: 低（手動でも十分）

2. スケジュール実行（タスクスケジューラ）

何が変わる？

毎朝決まった時刻に自動でパイプライン実行。

メリット

- 担当者が朝一番に「コマンド叩く」作業がなくなる
- WP 編集に集中できる

実装イメージ

```
Windows タスクスケジューラで:
  - トリガー: 毎日 9:00
  - 操作: powershell -Command "cd projects\minpaku-blog ; .venv\Scripts\python.exe main.py"
```

優先度: 中

3. 旧 `steps/dify.py` 削除

何が変わる？

不要になったコードがリポジトリから消える。

メリット

- コード可読性向上
- 「どっちが本物？」の混乱がなくなる

優先度: 低（残しておいても害はない）

4. はてなブログ自動投稿（AtomPub API）

何が変わる？

WP で公開後、はてなブログにも自動転載される。

メリット

- はてなブックマークからの流入が期待できる
- API 公式・無料

実装イメージ

```
# 新ファイル: steps/hatena.py
# はてな AtomPub API（XML）でPOST
```

優先度: 中（運用効果次第）

中期で検討（1〜2週間）

5. 知識ベースの ChromaDB 化

何が変わる？

知識ベースが大きくなった時（30,000 トークン超）に、ベクトル検索で必要部分だけ取り出す方式に切替。

トリガー条件

- `knowledge/` 配下の合計サイズが**全部合わせて 30K トークン**を超えた時
- または、過去記事が**50本以上**になった時

実装イメージ

```
# 新ファイル: steps/rag.py
# 1. knowledge/ の全ファイルを ChromaDB に投入
# 2. 記事生成時に「キーワードに関連する」上位 K 件だけを Claude に渡す
```

メリット

- KB が大きくてもコスト線形にしか増えない
- より関連性の高い情報だけが Claude に渡る

デメリット

- セットアップ複雑化
- ベクトル検索の精度調整が必要

6. 記事の質スコア推移を可視化

何が変わる？

毎回の評価スコアをスプレッドシートに記録 → グラフ化して品質トレンドを追跡。

メリット

- プロンプト改善の効果が定量で見える
- 質が悪化したら早く気づける

実装イメージ

スプレッドシートの列に `score_overall`, `score_depth` などを追加 → main.py 末尾で書き込み。

7. ~~画像生成統合~~（実装済み）

Flux 1.1 Pro + HTML文字入れで実装済み。詳細は 5.5 サムネイル生成 を参照。

今後の改善余地:
- ブランドカラー / フォントを `templates/thumbnail.html.tpl` で調整
- 記事カテゴリーに応じた色やレイアウトのバリエーション追加

大規模リファクタ案（必要になったら）

8. Ahrefs API 完全自動化

トリガー条件

- 月 200 記事以上の運用が見えてきた時
- または「毎日新鮮なキーワードを補充したい」要件が出た時

コスト試算

構成	月コスト
Lite + 半自動（現状）	$129
Standard + API 自動化	$249 + 追加ユニット代
Advanced + API 自動化	$449 + 追加ユニット代

9. 評価結果のフィードバックループ

何が変わる？

評価で繰り返し指摘される弱点を、自動で `generate_article.md` プロンプトに反映する仕組み。

イメージ

```
1. 直近10記事の評価で「独自性スコアが平均6.5/10」と判明
2. 自動で generate_article.md に「独自性をさらに重視」を追加
3. 翌日からの記事で改善されるか観察
```

注意

完全自動化すると「プロンプトが暴走」する危険があるので**人間承認ステップ必須**。

10. マルチプロジェクト化

何が変わる？

民泊以外のブログ（観光・ビジネス・etc.）にも同じ仕組みを横展開。

既に対応済み

このリポジトリは `projects/` 配下に複数プロジェクトを持てる構成。

```
projects/
├── minpaku-blog/      ← 現プロジェクト
└── tourism-blog/      ← 将来追加例
```

`shared/` フォルダに共通ロジックを抽出することも可能（YAGNI で今は未実装）。

本手順書のメンテナンス

このドキュメント自体も**定期更新**が必要です。

イベント	このドキュメントを更新
新しい外部 API を追加	12. 付録 の API 一覧に追加
評価軸を変更	5.3 評価 に反映
新しいエラーパターン発見	10. トラブルシューティング に追加
プロンプト大改訂	4. プロンプト改善ガイド に反映
Ahrefs プラン変更	7. コスト・運用費 を更新

---

10. トラブルシューティング

担当者が「あれ？」となった時の自己解決ガイド。

main.py 実行時の代表的エラー

エラー1: `ANTHROPIC_API_KEY が .env にありません`

原因

- `.env` ファイルが存在しない
- API キーが空 or プレースホルダーのまま
- システムに同名の空環境変数があり .env を上書きしていない

対処

```
1. projects/minpaku-blog/.env が存在するか確認
2. ANTHROPIC_API_KEY=sk-ant-... が正しく設定されているか確認
3. main.py は load_dotenv(override=True) で .env 優先になっているので、
   .env を編集すれば直る
```

エラー2: `gspread.exceptions.SpreadsheetNotFound`

原因

- スプレッドシート ID が間違っている
- サービスアカウントが共有されていない

対処

```
1. .env の KEYWORDS_SHEET_ID を確認（URL の /d/ と /edit の間）
2. スプレッドシートを「共有」→ サービスアカウントのメール（google-credentials.json の client_email）を編集者で追加
```

エラー3: `WordPressError: 401 Unauthorized`

原因

- アプリケーションパスワードが正しくない
- WP_USERNAME が間違っている

対処

```
1. WP管理画面でアプリケーションパスワードを再生成
2. .env の WP_APP_PASSWORD を更新
3. WP_USERNAME がログイン名と一致しているか確認
```

エラー4: `EvaluationError: JSON パース失敗`

原因

Claude が返した JSON が壊れている（稀）。

自動対処

- `json-repair` ライブラリで自動修復を試みる
- それでもダメなら評価をスキップして WP 投稿は続行
- `outputs/*.evaluation_error.txt` にエラー詳細が残る

担当者の対処

通常は何もしなくてOK（パイプラインは続行している）。エラーが頻発するならプロンプト見直し。

エラー5: 文字化け（日本語が ??? になる）

原因

Windows のターミナルが cp932 エンコーディング、Python が UTF-8 を出力。

対処

各 Python スクリプトの先頭で `sys.stdout.reconfigure(encoding="utf-8")` を実行済みなので通常は出ない。

それでも出る場合:

```
chcp 65001  # PowerShell を UTF-8 モードに
```

エラー6: Claude API レートリミット

原因

短時間に大量リクエストして上限を超えた。

対処

- 数分待って再実行
- 連続実行する場合は間隔を空ける（例: 30秒に1記事）
- アカウント設定で Rate Limit 引上げ申請可能

ログの見方

main.py の出力例:

```
スプレッドシートから未処理キーワードを取得...
対象: 行 5 / キーワード '民泊 京都 古民家'

[1/6] Claude API で記事生成中...
  完了（108.4秒 / in=165 out=5,705 cache作成=3,951 cache読込=0）
  バックアップ: outputs/2026-05-04_174928_民泊とは.md

[2/6] Claude + web_search で評価中...（30〜90秒）
  完了（83.2秒 / in=33,026 out=4,070 web検索=1 回）
  総合スコア: 7.8 / 10
  内訳: 網羅=8 / 深さ=9 / 独自=8 / 実用=9 / 読み易=7 / 検索意図=6

[3/6] 加筆マーカー埋め込み中（ヒント 5 件）...
  完了（25.4秒 / マーカー 5 個 / in=7,024 out=6,413）
  注釈版: outputs/2026-05-04_174928_民泊とは.annotated.md

[4/6] サムネイル生成中...（30〜60秒）
  完了（45秒）
  サムネ: outputs/2026-05-04_174928_民泊とは.thumbnail.png
  WP メディア ID=618

[5/6] WordPress に下書き投稿中...
  下書きID=619 / アイキャッチ設定: メディア ID=618
  ...

[6/6] スプレッドシート更新...
```

項目	意味
`in=165`	入力トークン数
`out=5,705`	出力トークン数
`cache作成=3,951`	キャッシュ作成（5分有効）
`cache読込=0`	キャッシュヒット（2回目以降増える）
`web検索=1 回`	web_search ツール呼び出し回数

緊急時対応

パイプラインが完全に動かない

```
1. git status で .env や .python-version が変わっていないか確認
2. git stash で変更を退避（or git checkout -- . で破棄）
3. .venv を再構築:
   rm -rf .venv
   python -m venv .venv
   .venv/Scripts/python.exe -m pip install -r requirements.txt
4. 動作確認
```

大量に間違った記事を WP に投稿してしまった

```
1. WP 管理画面 → 投稿 → 状態「下書き」フィルタ
2. 一括選択 → 一括操作「ゴミ箱に移動」
3. ゴミ箱からも削除（完全削除）
4. スプレッドシートで該当行の status を空に戻す
   → main.py 再実行で再生成可能
```

API キーが漏えいした疑い

```
ANTHROPIC: console.anthropic.com で当該キーを Revoke → 新キー発行 → .env 更新
GOOGLE   : Cloud Console で Service Account のキーを Disable → 新キー発行
WP       : 管理画面でアプリケーションパスワードを「取り消す」 → 再生成
```

---

11. 用語集

用語	意味
API	Application Programming Interface。プログラムから外部サービスを呼ぶ仕組み
API キー	API を使うための「鍵」。漏らすと悪用される
CSE	Custom Search Engine（Google の検索 API）
CSV	Comma-Separated Values。表形式のテキストファイル
E-E-A-T	Experience（経験）・Expertise（専門性）・Authoritativeness（権威性）・Trustworthiness（信頼性）。Google が品質評価で重視
KD	Keyword Difficulty。Ahrefs のキーワード難易度指標（0-100）
KW	Keyword の略
LLM	Large Language Model。Claude や GPT などの大規模言語モデル
OAuth	サードパーティ認証の仕組み。「Google でログイン」など
RAG	Retrieval-Augmented Generation。LLM に外部知識を参照させて精度を上げる手法
SaaS	Software as a Service。月額課金型の Web サービス
SDK	Software Development Kit。プログラムから API を簡単に叩くためのライブラリ
SEO	Search Engine Optimization。検索エンジンで上位表示する施策
SERP	Search Engine Results Page。検索結果ページ
Service Account	Google Cloud の「自動化用 Google アカウント」。人間アカウントと別
AtomPub	Atom Publishing Protocol。はてなブログが採用している XML ベースの API 規格
Application Password	WordPress の API 用パスワード。通常のログインパスワードと別管理
Hallucination（ハルシネーション）	LLM が事実でない内容を生成する現象
Markdown	見出しや太字を `# 太字` のような記号で表現するテキスト記法
Prompt（プロンプト）	LLM への指示文
Prompt Cache	同じプロンプトを5分以内に再使用する時に料金を90%割引する仕組み
Tool Use	LLM が外部ツール（web_search 等）を使えるようにする機能
Token（トークン）	LLM が文章を処理する単位。日本語は約2文字 = 1 トークン

用語	意味
venv	Virtual Environment。プロジェクト専用の Python 環境
Webhook	サーバー間で「イベント発生時に通知する」仕組み

---

12. 付録

全環境変数一覧

`projects/minpaku-blog/.env` に設定する全変数。

変数名	値の例	説明
`ANTHROPIC_API_KEY`	`sk-ant-api03-xxx...`	Claude API 認証キー
`CLAUDE_MODEL`	`claude-sonnet-4-5`	使用する Claude モデル ID
`REPLICATE_API_TOKEN`	`r8_xxx...`	Replicate API トークン（Flux 1.1 Pro 経由でサムネ画像生成）
`KEYWORDS_SHEET_ID`	`1Nm5iVOYYr-...`	キーワード一覧スプレッドシートの ID
`WP_SITE_URL`	`https://example.com/`	WordPress サイトの URL
`WP_USERNAME`	`admin`	WordPress ログイン名
`WP_APP_PASSWORD`	`xxxx xxxx xxxx ...`	WordPress アプリケーションパスワード（24文字）
`DIFY_API_KEY`	`app-xxx...`	（旧）Dify API キー・現在は使用していない
`DIFY_API_BASE_URL`	`https://api.dify.ai/v1`	（旧）Dify API URL

全ファイル一覧（重要なものだけ）

パス	役割	Git管理
`main.py`	メイン実行スクリプト	✅
`replenish_keywords.py`	キーワード補充スクリプト	✅
`extract_kb.py`	知識ベース抽出スクリプト	✅
`requirements.txt`	Python パッケージ一覧	✅
`.env`	API キー	❌
`.env.example`	.env の雛形	✅
`google-credentials.json`	Google 認証情報	❌
`steps/claude.py`	Claude 呼び出し	✅
`steps/evaluate.py`	評価ロジック	✅
`steps/annotate.py`	加筆マーカー埋め込み	✅
`steps/thumbnail.py`	サムネ生成（Flux + 文字入れ）	✅
`steps/sheets.py`	Sheets 操作	✅
`steps/wordpress.py`	WP 操作（メディアアップロード含む）	✅
`prompts/generate_article.md`	記事生成プロンプト	✅
`prompts/evaluate_article.md`	評価プロンプト	✅
`prompts/annotate_article.md`	加筆マーカープロンプト	✅
`prompts/generate_thumbnail_prompt.md`	サムネ画像プロンプト生成	✅
`prompts/extract_voice_guide.md`	voice 抽出プロンプト	✅
`prompts/extract_industry_facts.md`	facts 抽出プロンプト	✅
`templates/thumbnail.html.tpl`	サムネ文字入れ HTML テンプレ	✅
`knowledge/voice_guide.md`	ブランドボイス	✅
`knowledge/target_audience.md`	ターゲット読者	✅
`knowledge/industry_facts.md`	業界事実	✅
`knowledge/personal_experiences.md`	実体験	✅
`knowledge/unique_perspectives.md`	独自視点	✅

パス	役割	Git管理
`knowledge/past_articles/*.md`	過去ベスト記事	✅
`inbox/*.csv`	Ahrefs CSV（投入待ち）	❌（gitignored）
`outputs/*.md`	生成記事バックアップ	❌
`.venv/`	Python 仮想環境	❌

外部 API 一覧

サービス	エンドポイント	認証	公式ドキュメント
Anthropic	`https://api.anthropic.com/v1/messages`	API キー	https://docs.anthropic.com/
Anthropic web_search	tools 経由	同上 API キー	https://docs.anthropic.com/en/docs/build-with-claude/tool-use/web-search-tool
Replicate（Flux 1.1 Pro）	`replicate.run('black-forest-labs/flux-1.1-pro', ...)`	API トークン	https://replicate.com/black-forest-labs/flux-1.1-pro
Google Sheets	gspread 経由	Service Account JSON	https://docs.gspread.org/
WordPress	`https://[サイト]/wp-json/wp/v2/posts`	App Password	https://developer.wordpress.org/rest-api/
WordPress Media	`https://[サイト]/wp-json/wp/v2/media`	App Password	https://developer.wordpress.org/rest-api/reference/media/

参考リンク

公式ドキュメント

- Claude API — 記事生成・評価
- Google Sheets API — キーワード管理
- gspread（Python） — Sheets ラッパー
- WordPress REST API — 投稿
- Ahrefs API — 将来の自動化候補
- はてなブログ AtomPub — 拡散候補

参考にすべき情報源

- Google Search Quality Rater Guidelines — E-E-A-T を本気で意識するなら必読

- Anthropic プロンプトエンジニアリング — プロンプト改善ガイド

このシステムについての連絡先

用途	連絡先
GitHub リポジトリ	https://github.com/hiro-57/content-automation-pipeline
Anthropic アカウント管理	https://console.anthropic.com/
Google Cloud 管理	https://console.cloud.google.com/
Ahrefs サポート	https://help.ahrefs.com/
WordPress サポート	サイト管理者

改訂履歴

日付	変更内容	担当
2026-05-04	初版作成	（初代担当者）

> このドキュメントは生きた文書です。**運用しながら気づいた改善点・発見・落とし穴を都度追記**してください。
> 「次の担当者が読んだ時に同じトラブルで悩まないように」が目標です。